ALGONQUIN COLLEGE

SCHOOL OF ADVANCED TECHNOLOGY

ICT - Applications & Programming
Computer Engineering Technology – Computing Science



A3

Scanner Implementation

The Organ: a TXT-DFA Scanner for the BioCompiler

Track A: Python Simulation • Track B: Domain-Specific Language (DSL)

Specification

ALGONQUIN COLLEGE



ASSIGNMENT 3 – LEXICAL ANALYZER (Scanner / Organ)

General View

Due Date: prior or on **Nov 1st 2026 (midnight)**.

- **2nd Due date** (until **Nov 8th 2026**) – **50%** off.

Earnings: **10%** of your course grade.

Development: Activity can be done **individually** or in teams (**only 2 students** allowed, from the same lab section).

Purpose: Development of a Scanner – the Organ of the BioCompiler

- We are progressively implementing the front-end compiler for **your language** (see your **A1**). This activity uses the **Cell** (coder, **A1**) and the **Tissue** (reader / buffer, **A2**) and adds the third stage: the **Organ** (scanner, **A3**). It is also an exercise in "*defensive programming*".
- The Organ is a **table-driven scanner**: its automaton is not hard-coded in C, it is loaded at run time from a **TXT file** (`compiler_scanner_dfa.TXT`) that describes the **TD** (Transition Diagram) you designed from your **RE** (Regular Expressions). The RE and the TD **must be designed BEFORE coding**.
  - You will use advanced data types (unions, enumerations, structures), *function dispatch* by action code, dynamic memory, and simple file input/output. The functions you write here will be used by the **System** (parser, **A4**) and the **Body** (runner, **A5**).
- The code must follow the *camelCase* style used in the provided files.
- Download the starter project **biocompiler.zip** (see A3 Assignment on Brightspace).

Preliminary Note 1: The two tracks of this course

This term, every student / team develops ONE language from A1 to A5, choosing one of two tracks:

- **Track A – Python Simulation:** your language is a well-defined **subset of Python 3**. Real Python programs written in that subset must be scanned correctly, including **NEWLINE**, **INDENT** and **DEDENT** tokens.
- **Track B – Domain-Specific Language (DSL):** you design a small language for a **domain of your choice** (recipes, robotics, music, smart home, finance, games, …) with its own keywords, typed identifiers and at least one domain-specific literal.

Keep the track chosen in A1/A2. Changing the track requires your professor's approval. Re-using Sofia or the Bio sample language as your language is not accepted (zero marks for "language adaptation").

1. About the ZIP Provided: the BioCompiler

The starter project (**biocompiler**) is a complete, working pipeline for the sample **Bio** language ("Genoma"). It compiles without warnings (`gcc -Wall`), all self-tests pass (option `8` runs 25 Organ

checks), and the seven example programs (`InputExample0-6.txt`) are scanned with **0 errors**, while `ErrorExample1.txt` shows the lexical errors. Your job is to transform it into the scanner of **your** language.

1.1 The biological analogy

Each stage of the compiler is named after a level of biological organization: simple units are combined into structures of increasing complexity, and each level only "talks" to the level directly below it.

Stage	Bio name	Classic name	Files	What it does
1	**Cell**	Coder	`Step1Cell.*`, `Main1Cell.c`	Encodes / decodes the source file (cipher key = language name).
2	**Tissue**	Reader (buffer)	`Step2Tissue.*`, `Main2Tissue.c`	Buffered reader: stores characters ("cells") and serves them one at a time.
3	**Organ**	**Scanner**	`Step3Organ.*`, `Main3Organ.c`	**This assignment.** Reads the Tissue and produces **Signals** (tokens), driven by the TXT DFA.
4	System	Parser	(A4)	Grammar-file driven parser (`compiler_parser_grammar.txt`).
5	Body	Runner	(A5)	Config-file driven runner (`compiler_runner_config.txt`).

Classic compiler term	BioCompiler term used in the code
Token	`Signal` (struct with `code` and `attribute`)
Token codes (enum TOKENS)	`enum SIGNALS` (e.g. `KW_S`, `MNID_S`, `STR_S`, `SEOF_S`)
tokenizer() / scanner function	`organScan()` – returns ONE Signal per call
Accepting-state functions (funcXXX)	`signalIdentifier`, `signalKeyword`, `signalIntegerLiteral`, `signalFloatLiteral`, `signalStringLiteral`, `signalComment`, `signalError`
Transition table and keyword table	`compiler_scanner_dfa.TXT` ([DFA] and [KEYWORDS]), loaded by `loadScannerDFA()`
nextClass() / nextState()	`charClass()` / `nextState()`
Reader buffer	`TissuePointer` (`tissueGetCell`, `tissueRetract`, `tissueAddCell`, …)
Token statistics	`organHistogram` printed by `printOrganData()`

1.2 Files of the Organ in biocompiler.zip

File	Role	Change it?
`Biology.h` / `Biology.c`	Language name (`STR_CompilerName`), typedefs, options, default file names, `main()`.	**TODO**: language name and typedefs
`Step1Cell.*`, `Step2Tissue.*`	Coder (A1) and reader (A2).	Keep (your own versions)
`Step3Organ.h`	Signals, attributes, operators, limits (`VID_LEN`, `ERR_LEN`, `NUM_LEN`, `MAX_LEXEME`, `BIO_INT_MAX`, `CHAR_CLASSES`), DFA constants, prototypes. **No keyword list**: keywords live in the TXT.	**TODO**: adapt to your language
`Step3Organ.c`	TXT loader, `charClass`, `nextState`, `startOrgan`, `organScan`, accepting functions, statistics, the signal table.	**TODO**: adapt to your language

File	Role	Change it?
`Main3Organ.c`	Driver: loads the DFA, decodes and loads the source, prints every Signal (operators as symbols, strings with their content) and the string table.	**TODO**: `printSignal` for your Signals
`Test3Organ.c`	Self-tests of the Organ (option `8`).	**TODO**: tests for your tokens
`compiler_scanner_dfa.TXT`	The language data: [CHAR_CLASSES], [KEYWORDS] and [DFA].	**TODO**: your DFA and keywords
`InputExample0-6.txt`, `ErrorExample1.txt`	Valid Bio programs (0 = empty file) and a file with lexical errors.	Replace with your inputs
`runbio3_organ.bat`, `runbio3_organ_complete.bat`, `clean.bat`	Build-and-test scripts (Section 6) / clean-up script.	**TODO**: adapt
`logo.txt`	ASCII logo printed by the program.	Use your own logo

Preliminary Note 2: Implementation notes

The provided code is functional but complete ONLY for the Bio "Hello World". You must continue the development so that it obeys YOUR specification (A1).

Using any language definition other than your own (Sofia, Bio, another team's language) is considered plagiarism.

The sample DSL (Bio) is a reference for Track B and a counter-example for Track A: Python has no identifier suffixes, uses # comments only to the end of the line, and gives meaning to newlines and indentation.

2. The Organ: General View

Note 1: About the Scanner (Organ)

The scanner reads the source program from the Tissue (reader / buffer) and produces a stream of token representations (Signals). It does not need to produce all the tokens before the next phase: in almost all compilers it is a function that returns ONE token each time the parser asks for it.

Our Organ is a mixture of two techniques:

- **Part 1 – token-driven:** a `switch` in `organScan()` recognizes simple tokens directly (delimiters and operators such as `(`, `;`, `+`, `++`, `==`, `<>`).
- **Part 2 – table-driven:** everything else (identifiers, keywords, numbers, strings, comments, domain literals) is recognized by simulating the DFA loaded from the TXT file, and the accepting state's **ACTION** selects the `signal…` function that builds the Signal.

- The input of the Organ is your source program, encoded by the Cell and loaded into a Tissue. The output of each call to `organScan()` is a **single Signal**.
- The table-driven part must recognize at least: identifiers (including keywords), numeric literals (integer and floating-point), string literals and comments – plus the track-specific elements of Section 4.
- To build the TXT you transform the lexical rules of your language into **regular expressions**, then into a **transition diagram**, then into the transition table. The state numbers in the TXT **must match** the numbers in your diagram.

2.1 The DFA file: compiler_scanner_dfa.TXT

The file has three sections. **[CHAR_CLASSES]** names the columns (their number must equal `CHAR_CLASSES` = 12, or the file is rejected); **[KEYWORDS]** lists the reserved words, one per line – they are loaded into `keywordTable`, and a keyword code is its **position** in this list; **[DFA]** has one row per state: fields are separated by `|` (so commas are free inside fields); lines starting with `#` and blank lines are ignored. Each row is: **STATE | one column per character class | STATE_TYPE | ACTION**.

Excerpt of compiler_scanner_dfa.TXT (30 keywords, 18 states × 12 classes)

```
[CHAR_CLASSES]
L|D|U|P|I|F|S|M|Q|C|E|O

[KEYWORDS]
# domain / structure
bio_data
bio_code
bio_int
...
pow
fmod

[DFA]
# STATE|  L|  D|  U|  P|  I|  F|  S|  M|  Q|  C|  E|  O|STATE_TYPE|ACTION
      0|  1|  7| 16| 16| 16| 16| 16| 16| 12| 14| 16| 16|      NOFS|NULL
      1|  1|  1|  1|  6|  2|  3|  4|  5|  6|  6|  6|  6|      NOFS|NULL
      2| 18| 18| 18| 18| 18| 18| 18| 18| 18| 18| 18| 18|      FSNR|IDENTIFIER
      6| 18| 18| 18| 18| 18| 18| 18| 18| 18| 18| 18| 18|      FSWR|KEYWORD
      7|  8|  7|  8|  9|  8|  8|  8|  8|  8|  8|  8|  8|      NOFS|NULL
      8| 18| 18| 18| 18| 18| 18| 18| 18| 18| 18| 18| 18|      FSWR|INTEGER
     12| 12| 12| 12| 12| 12| 12| 12| 12| 13| 12| 16| 12|      NOFS|NULL
     13| 18| 18| 18| 18| 18| 18| 18| 18| 18| 18| 18| 18|      FSNR|STRING
     16| 18| 18| 18| 18| 18| 18| 18| 18| 18| 18| 18| 18|      FSNR|ERROR
     17| 18| 18| 18| 18| 18| 18| 18| 18| 18| 18| 18| 18|      FSWR|ERROR
```

Element	Meaning in the provided file
[KEYWORDS]	30 unique words (`bio_data`, `bio_code`, `bio_int`, `bio_flt`, `bio_str`, `if`, `then`, `else`, `while`, `do`, `return` and 19 math functions). A word without a type suffix that is not in this list is a lexical error.
Character classes	`L`=letter, `D`=digit, `U`=_ , `P`=. , `I`=$, `F`=% , `S`=@ , `M`=& , `Q`=" , `C`=# , `E`=end of source, `O`=other. Mapped by `charClass()` in `Step3Organ.c`.
Transition values	0–17 = next state; **16 = ESNR** (error, no retract); **17 = ESWR** (error, with retract); **18 = FS** (final state reached).
STATE_TYPE	`NOFS` = not final (keep reading); `FSNR` = final, keep the last char; `FSWR` = final, **retract** the last char.
ACTION	`IDENTIFIER`, `KEYWORD`, `INTEGER`, `FLOAT`, `STRING`, `COMMENT`, `ERROR` (or `NULL`). `parseAction()` maps it to `DFAAction`, and `organScan()` calls the matching `signal…` function.
Bio identifiers	A letter followed by letters/digits/_ and ending with a **type suffix**: `$` integer (`IVID_S`), `%` float (`FVID_S`), `@` string (`SVID_S`), `&` method name (`MNID_S`). Without a suffix the lexeme ends in state 6 and is checked against the keyword table.
Bio comments / strings	Comments are delimited `# … #` (states 14–15); strings `" … "` (states 12–13). Reaching end of source inside them goes to the error state.

Note 2: Keep the C code and the TXT consistent

The TXT is data, but some of its assumptions are also written in C. When you change the DFA, keep these in agreement:

- `CHAR_CLASSES` (Step3Organ.h) = number of columns of [CHAR_CLASSES] and of every [DFA] row; the loader uses it for every column and rejects a mismatching header.

- `charClass()` must return one column index per class, in the same order as [CHAR_CLASSES].
- `ESNR`, `ESWR` and `FS` must match the error/final rows of your TXT (`nextState()` returns `ESNR` for invalid input).
- Keyword codes are positions in [KEYWORDS]; the parser (A4) finds `KW:name` in this same table, so reordering keywords needs no change in C.
- Limits: `MAX_STATES_` = 64, `MAX_CLASSES_` = 16, `MAX_KEYWORDS` = 64.

3. Implementation Overview

The Organ is composed of the following functions (all in `Step3Organ.c` unless noted):

Function	Status	Responsibility
`loadScannerDFA(TXTFile)`	Done	Reads [CHAR_CLASSES] (checked), [KEYWORDS] and [DFA]; frees the keywords of a previous load.
`startOrgan(buffer)`	Done	Defensive start: checks buffer and DFA, resets position, `line = 1`, statistics.
`organScan()` – Part 1	**TODO** (adapt)	`switch` for tokens not described by the DFA: `( ) { } , ;`, `+ - * / ^`, `++`, `=`/`==`, `<`/`<=`/`<>`, `>`/`>=`, `&&`, `||`, `!` (look-ahead + `tissueRetract`).
`organScan()` – Part 2	Done (adapt)	DFA simulation; lexemes up to `MAX_LEXEME`; retract on `FSWR`; dispatch by ACTION; records `lexemeLine` (line where the Signal starts).
`charClass(c)` / `nextState(s, c)`	**TODO** (adapt)	Your character classes, in [CHAR_CLASSES] order.
`signalIdentifier`, `signalKeyword`	Done (adapt)	Suffix classification, `VID_LEN` check, keyword lookup; unknown words are counted errors.
`signalIntegerLiteral`, `signalFloatLiteral`	Done (adapt)	Range checks (`NUM_LEN`, 0…`BIO_INT_MAX`, 4-byte float).
`signalStringLiteral`	Done	Stores the content WITHOUT quotes in `stringLiteralTable`; `RTE_S` if the table cannot grow.
`signalComment`, `signalError`	Done	Error lexemes cut to `ERR_LEN-3` + `...`, on one line, counted in `numOrganErrors`.
your `signal…` functions	**TODO**	One accepting function per new kind of lexeme (domain literal, Track A NEWLINE/INDENT/DEDENT support).
`printSignal` (`Main3Organ.c`)	Done (adapt)	Prints every Signal with its attribute.

- Constants: `VID_LEN` (identifier length), `ERR_LEN` (error lexeme length), `NUM_LEN` (maximum digits of a number), `RTE_CODE` (run-time error value), `KWT_SIZE` (number of keywords loaded from the TXT), `MAX_KEYWORDS`, `NUM_SIGNALS`.

Note 3: About error handling

There is ALWAYS an output: the correct sequence of Signals, or Signals mixed with errors. The Organ must detect the error, report it with its line, and CONTINUE scanning (error recovery).

- **Error lexemes:** if the erroneous lexeme is longer than `ERR_LEN` characters, store only the first `ERR_LEN-3` characters followed by three dots (`...`). (The provided `signalError` only truncates – fix it.)

- **Run-time errors:** when it is not possible to add a character to the `stringLiteralTable`, set the global `errorNumber` to `RTE_CODE` and return an `RTE_S` Signal whose attribute is the string "Run Time Error:".
- **Lexical errors to handle at least:** illegal characters, unterminated strings, unterminated comments (DSL block comments), malformed numbers, numbers out of range, identifiers longer than `VID_LEN`.

The number of errors is counted in `numOrganErrors`, printed at the end, and returned as the exit code of option 3 (EXIT_FAILURE when there are errors).

Note 4: Robustness built into the provided Organ (keep it in your version)

- Keywords are unique and come from the TXT; a word without suffix that is not a keyword is a counted error.
- String literals are stored without their quotes; lexemes longer than `MAX_LEXEME` (1024) are errors (never silently cut).
- Integer literals: at most `NUM_LEN` digits and ≤ `BIO_INT_MAX` (32767); floats must fit in 4 bytes; identifiers have at most `VID_LEN` characters.
- Errors are reported at the line where the lexeme STARTS (an unterminated string is reported where it was opened).
- A keyword or identifier at the very end of the file is accepted (the DFA moves to state 6 on end of source).
- The relational operators `<=` and `>=` and the logical operators `&&`, `||` and `!` are recognized.

4. Track Requirements

The minimum lexical content of your language depends on your track. Everything listed here must be present in your **RE/TD document**, your **TXT**, your **code** and your **input files**.

4.1 Track A – Python Simulation

Your language (give it a name, e.g. *Viper* – not "Python") is a subset of Python 3: any valid program of your subset must also be a valid Python program, and it must be tokenized as Python would tokenize it.

Category	Minimum requirement (Track A)
Identifiers	`[A-Za-z_][A-Za-z0-9_]*` – **no suffixes**. Case sensitive. Keywords are identifiers found in the keyword table → `KW_S`; others → `ID_S` (single identifier Signal).
Keywords	`False None True and break continue def elif else for if in is not or pass return while` (17). Optional: `import from class lambda global try except`. Built-ins such as `print`, `range`, `len` are **identifiers**, not keywords.
Integer literals	Decimal `[0-9]+` with range check. Optional (bonus): `0x…`, `0b…`, underscores (`1_000`).
Float literals	`[0-9]+.[0-9]+` and `.5` / `5.` forms. Optional (bonus): exponent (`1e-3`).
Strings	`'…'` and `"…"` on one line, with escapes `\' \" \\ \n` not ending the string. Unterminated string → error. Optional (bonus): triple-quoted strings.
Comments	`#` up to the **end of the line** (NOT `# … #` as in Bio). A comment may be emitted as `CMT_S` or discarded – document your choice.
NEWLINE	`NEWLINE_S` at the end of each logical line. Blank lines and comment-only lines produce no NEWLINE. Optional (bonus): no NEWLINE inside `( )`, `[ ]`, `{ }` (implicit line joining).

Category	Minimum requirement (Track A)
INDENT / DEDENT	At the start of each logical line compare the indentation with a **stack** of levels: deeper → one `INDENT_S`; shallower → one `DEDENT_S` per popped level; a level not on the stack → error ("inconsistent dedent"). At end of source emit the pending DEDENTs before `SEOF_S`. Define and document how a tab counts (e.g. 4 spaces) and whether mixing tabs/spaces is an error.
Operators	`+ - * / // % ** = == != < <= > >= += -=` (logical operators are the keywords `and or not`).
Delimiters	`( ) [ ] { } , : .` (the semicolon is optional in Python; `;` may be accepted as `EOS_S`).

Python changes the structure of organScan()

In Bio, organScan() skips spaces, tabs AND newlines before every token. In Python the newline and the indentation ARE tokens:

- stop skipping `\n`: produce `NEWLINE_S` (unless the line was blank/comment-only or you are inside brackets);
- after a NEWLINE, measure the indentation of the next non-blank line and emit INDENT/DEDENT – use the provided `LanguageAttributes` structure (`indentationCharType`, `indentationCurrentPos`) and extend it with a stack and a counter of pending DEDENTs (one call to `organScan()` returns only ONE Signal);
- the DFA must end identifiers on any character that is not `[A-Za-z0-9_]`, comments on `\n`, and accept both quote characters.

Example input (Track A)

```
# factorial in Viper (Track A example)
def fact(n):
    if n <= 1:
        return 1
    return n * fact(n - 1)

print("5! =", fact(5))
```

Expected Signal stream (illustrative – `organScan()` returns ONE Signal per call; follow the numbers, column by column):

#	Signal	Attribute	#	Signal	Attribute	#	Signal	Attribute
1	CMT_S		15	NEWLINE_S		29	RPR_S	
2	KW_S	def	16	INDENT_S		30	NEWLINE_S	
3	ID_S	fact	17	KW_S	return	31	DEDENT_S	
4	LPR_S		18	INL_S	1	32	ID_S	print
5	ID_S	n	19	NEWLINE_S		33	LPR_S	
6	RPR_S		20	DEDENT_S		34	STR_S	0 ("5! =")
7	COL_S		21	KW_S	return	35	COM_S	
8	NEWLINE_S		22	ID_S	n	36	ID_S	fact
9	INDENT_S		23	ART_OP_S	*	37	LPR_S	
10	KW_S	if	24	ID_S	fact	38	INL_S	5
11	ID_S	n	25	LPR_S		39	RPR_S	
12	REL_OP_S	<=	26	ID_S	n	40	RPR_S	

#	Signal	Attribute	#	Signal	Attribute	#	Signal	Attribute
13	`INL_S`	`1`	27	`ART_OP_S`	`-`	41	`NEWLINE_S`	
14	`COL_S`		28	`INL_S`	`1`	42	`SEOF_S`	`0`

4.2 Track B – Domain-Specific Language (DSL)

You design a small language for a **domain** you know. It must look and feel specific to that domain (its keywords, literals and identifiers come from the domain vocabulary), and it must be rich enough to exercise a real automaton.

Category	Minimum requirement (Track B)
Name and domain	A language name (not Sofia / Bio, no `bio_` keywords) and a one-paragraph description of its domain and purpose in your A1.
Keywords	At least **12** keywords from the domain vocabulary (data types, control flow, domain actions).
Typed identifiers	At least **2 kinds** of identifiers that are distinguished **lexically** (prefix or suffix, like the Bio suffixes `$ % @ &`), each producing its own Signal.
Numeric literals	Integer and floating-point literals, with range checking.
String literals	At least one string form; unterminated string → error.
Domain literal	At least **1** literal specific to your domain recognized by the DFA with its own accepting function and Signal, e.g. time `07:30`, date `2026-11-01`, musical note `C#4`, colour `#1E90FF`, quantity `250g`, coordinate `45.4N`, dice roll `2d6`.
Comments	Line or block comments (Bio uses `# … #`); an unterminated block comment → error.
Operators	Assignment, arithmetic, relational and logical operators, including at least **2 multi-character** operators (e.g. `==`, `<>`, `++`, `->`, `:=`).
Delimiters	Grouping and statement delimiters appropriate to your syntax.

Domain idea	Example statement	Domain literal
Kitchen recipes	`step bake(oven@, 180c, 00:35);`	temperature `180c`, time `00:35`
Robot control	`move arm& to (12.5, 4.0) speed 30%;`	percentage `30%`
Music	`play melody$ [C4 E4 G4] tempo 120;`	note `C#4`
Smart home	`when door? opens after 22:00 do lights@ off;`	time `22:00`
Board games	`roll 2d6 -> score$;`	dice `2d6`

The Bio sample (`InputExample1.txt`) shows the expected **style** of a DSL program – typed identifiers (`Message@`, `main&`), sections (`bio_data`, `bio_code`) and `# … #` comments – but your language must have its own vocabulary and symbols.

InputExample1.txt (Bio reference program)

```
# Sofia Example #
bio_int main&() {
        bio data {
                bio str Message@;
        }
        bio_code {
                Message@ = "Hello, World!";
                print&(Message@);
                return 0;
        }
}
```

5. Implementation Steps

1. **Check the previous stages.** Your Cell (A1) and Tissue (A2) must work: options `6` and `7` (self-tests) must pass. **Problems in the reader affect all the next assignments.**
2. **Rename the language** in `Biology.h` (`STR_CompilerName` cannot be "Sofia" or "Bio") and adapt the typedefs to your prefix (e.g. `vip_intg`). The language name is also the **cipher key** of the Cell: after renaming, **re-encode your input files** (e.g. `biology 1 1 Input.txt CODED.TXT`), otherwise decoding produces garbage that looks like scanner errors.
3. **Design before coding:** write the RE of every lexeme handled by the DFA, draw the TD (number the states), and derive the transition table. Hand them in as a PDF (**A3_RE_TD.pdf**).
4. **Write your DFA** in `compiler_scanner_dfa.TXT`: one column per character class, the STATE_TYPE and the ACTION of every state. Update the header comments of the file.
5. **Update `Step3Organ.h`:**
   a. **TODO**: `enum SIGNALS`, `signalStrTable` and `NUM_SIGNALS` – one Signal per token class of your language (for instance, if all identifiers follow the same RE, use one identifier Signal). Track A adds `ID_S`, `NEWLINE_S`, `INDENT_S`, `DEDENT_S`, `COL_S`, `LBK_S`, `RBK_S`; Track B adds your typed-identifier and domain-literal Signals.
   b. **TODO**: attributes – `SignalAttribute` (union), operator enumerations (`AriOperator`, `RelOperator`, `LogOperator`) and `IdAttibutes`, using your typedefs.
   c. **TODO**: special characters (`…_CHR`), `ESNR` / `ESWR` / `FS` and the DFA constants (Note 2).
   d. **TODO**: your keywords in [KEYWORDS] of the TXT (unique, case-sensitive; at most `MAX_KEYWORDS`).
   e. **TODO**: `LanguageAttributes` (Track A: indentation stack, pending DEDENTs, bracket depth).
   f. **TODO**: new `DFAAction` values and prototypes for your new accepting functions.
6. **Update `Step3Organ.c`:**
   a. **TODO**: `charClass()` – your classes, in TXT column order; `parseAction()` – your new ACTION names.
   b. **TODO**: `organScan()` Part 1 – every token not described by the DFA (use look-ahead with `tissueGetCell` and give back with `tissueRetract`).
   c. **TODO**: `organScan()` Part 2 – dispatch of your new actions (Track A: newline/indentation logic before Part 1).
   d. **TODO**: all `signal…` functions (Section 3 and Note 4) using **defensive programming**.
7. **Update `Main3Organ.c`:** `printSignal()` must print every Signal of your language with its attribute.
8. **Update `Test3Organ.c`:** replace the Hello World checks by at least **15** checks on YOUR tokens (including at least 3 error cases). Option `8` must report "ALL TESTS PASSED".
9. **Create your inputs:** at least one valid program (≥ 30 lines, using every token of your language) and one **error file** with at least 5 different lexical errors. The scanner must survive the error file.
10. **Adapt the script** (Section 6) and check that the build has no warnings and that `err.txt` is empty for valid inputs.

Example – Part 1 of organScan() (provided code): look-ahead for two-character operators.

```
case '=': {
    bio char nx = tissueGetCell(organBuffer);   /* look ahead one character */
    if (nx == '=') {
        result.code = REL_OP_S;
        result.attribute.codeType = OP_EQ;     /* == */
    } else {
        tissueRetract(organBuffer);            /* give the character back */
        result.code = ASS_OP_S;                /* =  */
    }
    scData.organHistogram[result.code]++;
    return result;
}
```

Example – an accepting function with defensive programming (provided in `Step3Organ.c`; adapt the datatypes to yours). The value is converted with a type wider than the target, so overflow can be detected instead of silently truncated:

```
Signal signalIntegerLiteral(bio_strg lexeme) {
    Signal t;
    bio_long value;
    memset(&t, 0, sizeof(t));
    if (!lexeme || strlen(lexeme) > NUM_LEN) {        /* too many digits */
        return signalError(lexeme ? lexeme : "");
    }
    value = strtol(lexeme, NULL, 10);
    if (value < 0 || value > SHRT_MAX) {               /* outside your integer range */
        return signalError(lexeme);
    }
    t.code = INL_S;
    t.attribute.intValue = (bio_intg)value;
    return t;
}
```

Brief explanation: the Signal starts empty; the length of the lexeme is checked against `NUM_LEN`; the conversion uses a wider type (`bio_long`) so a value beyond the range of your integer type is detected instead of being truncated by a cast; negative numbers are not part of the literal (the minus sign is a unary operator); any problem is reported through `signalError`, which counts the error.

> ***Note 5: Language modification***
>
> *As in the previous assignments, your language definition can still evolve. You are free to modify it, but your A1 document, your RE/TD, your TXT, your code and your input files must always describe the SAME language.*

6. Script and Expected Output

The provided scripts build the project with **gcc** and run the Organ stage. Adapt them and submit them.

Script	What it does	Your adaptation
`runbio3_organ.bat`	Checks gcc and the DFA, compiles all files (`gcc -std=gnu99 -Wall … -o biology.exe -lm`), encodes `InputExample1.txt` and scans it (option `3`).	Your file names
`runbio3_organ_complete.bat`	Compiles, runs the self-tests `6`, `7`, `8`, encodes + scans **InputExample0–6** (must succeed) and **ErrorExample1** (must be rejected). Exit code = number of failures.	Your valid inputs and error file

Command-line usage

```
biology 1 1 <plain.txt> <encoded.txt>     (Cell: encode with the language-name key)
biology 3 <encoded.txt> [dfa.TXT]          (Organ: scan, default compiler scanner dfa.TXT)
biology 8 [dfa.TXT]                        (Organ self-tests)
```

Output of option 3 on InputExample1.txt (actual)

```
[Organ ] DFA loaded: 18 states x 12 classes, 30 keywords from 'compiler_scanner_dfa.TXT'
Reading file Example1_enc.txt ....Please wait

Scanning source file (Organ  - TXT DFA)...

Signal        Attribute
----------------------------------
CMT_S
KW_S            bio_int
MNID_S          main&
LPR_S
RPR_S
LBR_S
KW_S            bio_data
LBR_S
KW_S            bio_str
SVID_S          Message@
EOS_S
RBR_S
KW_S            bio_code
LBR_S
SVID_S          Message@
ASS_OP_S
STR_S           0    "Hello, World!"
EOS_S
MNID_S          print&
LPR_S
SVID_S          Message@
RPR_S
EOS_S
KW_S            return
INL_S           0
EOS_S
RBR_S
RBR_S
SEOF_S          0

Printing string table...
----------------------------------
Hello, World!\0
----------------------------------
Number of scanner errors: 0
----------------------------------
Statistics (Organ ):
----------------------------------
Signal[SVID_S]=3
Signal[MNID_S]=2
Signal[INL_S]=1
Signal[STR_S]=1
Signal[ASS_OP_S]=1
Signal[LPR_S]=2
Signal[RPR_S]=2
Signal[LBR_S]=3
Signal[RBR_S]=3
Signal[KW_S]=5
Signal[EOS_S]=4
Signal[CMT_S]=1
Signal[SEOF_S]=1
----------------------------------
```

Option 3 on ErrorExample1.txt (actual, error lines only)

```
ERR_S           99999
ERR_S           ?
ERR_S           ThisIdentifierIsMuchTooLong$
ERR_S           "unterminated;  } }
Number of scanner errors: 4
Signal[ERR_S]=4
```

- **About the executable:** its name does not need to match your IDE project. What matters is that the script compiles without errors (and warnings) and shows the list of Signals.

- **About inputs:** at least one valid input file and one error file written in **your** language (Track A: Python subset; Track B: your DSL). Bio or Sofia inputs are not accepted.
- **About err.txt:** for valid inputs, `err.txt` must be empty.

7. Submission Details

- **Digital submission:** compress into a **zip** file the complete project and attach it on Brightspace:
  - all `.c` / `.h` files, `compiler_scanner_dfa.TXT`, your **logo** and **script**;
  - your **input files** (valid + error file, plain and encoded) and the corresponding **outputs** (`out*.txt`, `err*.txt`);
  - **A3_RE_TD.pdf** – regular expressions, transition diagram and transition table matching the TXT;
  - a short **README** stating the track (A or B), the language name and how to run the script.
- **File name:** your **last name** followed by the last three digits of your student number, e.g. **A3_Sousa123.zip**; for teams: **A3_Sousa123_Melo456.zip**. Only students from the same lab section can form a team.
- Assignments will not be marked if there are no source files or input files in the submission. The submission must follow the course **Submission Standard and Marking Guide** (Brightspace → Assignments → Standards).
- **Evaluation note:** if your program compiles, runs, and produces correct output files, it is considered a *working program*. Your professor will also try to "crash" your functions with a modified main program and additional test files – test your code as much as you can!
- In case of emergency (Brightspace not working), submit your zip file by e-mail to your **lab professor**.

8. About Lab Demos

- **Main idea:** the demo during the lab sessions is **required** to get marks for this assignment; marks are reported in CSI.
- **How to proceed:** demonstrate your scanner to your lab professor (in the lab or in a private Zoom session) during the lab of the corresponding week. Teams must present together; only students who present receive the marks.
- Be ready to explain your RE, your TD, the TXT, and any part of the code.

9. Marking Rubric

Maximum deduction	Deduction event
—	**Plagiarism:**
10.0 pt	3-strike policy[1] (**AA32**, **SA07** and **IT01**)
—	**Severe errors:**
5 pt	Late submission (after the 2nd due date)
10.0 pt	Missing demo (zero[2])
CRITICAL	**Process**
10 pt (100%)	Does not compile with the submitted script (gcc) or Visual Studio 2022
5 pt (50%)	Compiles but crashes after launch (fix by debugging)
PATTERN	**Non-compliance**
10 pt (100%)	Track not respected: not a Python subset (A) or not a proper DSL (B); Sofia/Bio re-used
2.5 pt (25%)	Missing files[3]: own inputs (valid + error file), TXT, RE/TD document
1 pt	Missing or non-adapted script
BASIC	**Execution**
2 pt	Input files not written in the chosen track's language
2 pt	Track-specific elements missing or wrong (A: NEWLINE / INDENT / DEDENT; B: typed identifiers / domain literal)
1 pt	Common errors (EOF, unterminated tokens, token mismatching, invalid tokens[4])
ADDITIONAL	**Small problems**
0.1 pt each	Warnings
1.0 pt	Missing defensive programming
0.5 pt	Inconsistency between RE/TD, TXT and code
0.5 pt	Self-tests (option 8) not adapted to your language
0.5 pt	Other minor errors
BONUS	**Additional marks**
Up to 1.0	Interesting improvements.
Final mark	**Formula: 10 × ((100 – ∑ penalties + bonus) / 100) – max score 11%.**

File update: Oct 12th 2026.

Good luck with A3 – may your Organ produce healthy Signals!

[1]Plagiarism detection implies the "3-strike" policy: starting with ZERO, then moving to course failure or program cancellation (see https://www.algonquincollege.com/policies/).
[2]If a course requires demos, they are not optional. A student who does not demo their work receives a grade of 0 on that assessment, not a grade reduction.
[3]It is impossible to evaluate a scanner without proper input files. If no input is provided, a default input will be used for checking.
[4]Common examples: unterminated multi-line comments, unterminated strings, malformed numbers, illegal characters, bad dedents (Track A).